Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Fall, Year:2024), B.Sc. in CSE (Day)

Lab Report NO : 04
Course Title: Algorithm Lab
Course Code: CSE 206 Section: 223-D5

Lab Experiment Name: Implement merge sort and quick sort on array

Student Details

	Name	ID
1.	Md. Reahoon Zannah	223002038

Lab Date : 13/11/2024
Submission Date : 15/11/2024
Course Teacher's Name : Md. Sabbir Hosen Mamun

Lab Report Status

Marks: …… **Signature:.....**
Comments:..... **Date:.....**

1. TITLE OF THE LAB REPORT EXPERIMENT

Implement merge sort and quick sort on array

2. OBJECTIVES/AIM

The objective is to implement two common sorting algorithms—Merge Sort and Quick Sort—on an integer array in Java.

- Merge Sort divides the array into halves, recursively sorts each half, and then merges them. It has a time complexity of O(n log n) in both the average and worst cases.
- Quick Sort partitions the array around a pivot and recursively sorts the subarrays. Its average time complexity is O(n log n), but in the worst case, it can be $O(n^2)$.

The goal is to understand and implement these algorithms efficiently, compare their performance, and gain practical experience in sorting techniques.

3. Array

The array is: [38, 27, 43, 3, 9, 82, 10]

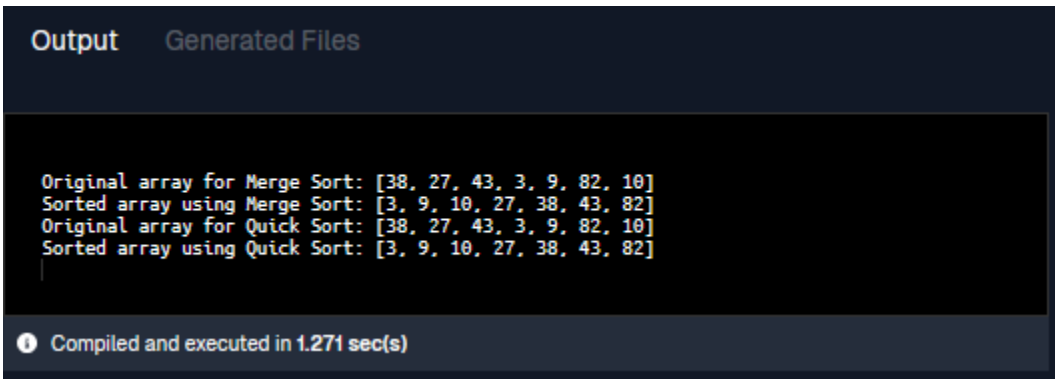
4. IMPLEMENTATION

```
import java.util.Arrays;
public class SortingAlgorithms {

    public static void mergeSort(int[] arr) {
        if (arr.length < 2) return;

        int mid = arr.length / 2;
        int[] left = Arrays.copyOfRange(arr, 0, mid);
        int[] right = Arrays.copyOfRange(arr, mid, arr.length);

        mergeSort(left);
        mergeSort(right);

        merge(arr, left, right);
    }
```

```
public static void merge(int[] arr, int[] left, int[] right) {
    int i = 0, j = 0, k = 0;
    while (i < left.length && j < right.length) {
        if (left[i] <= right[j]) {
            arr[k++] = left[i++];
        } else {
            arr[k++] = right[j++];
        }
    }
    while (i < left.length) arr[k++] = left[i++];
    while (j < right.length) arr[k++] = right[j++];
}

public static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

public static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}

public static void main(String[] args) {
    int[] arr1 = {38, 27, 43, 3, 9, 82, 10};
    System.out.println("Original array for Merge Sort: " + Arrays.toString(arr1));
```

```
        mergeSort(arr1);
        System.out.println("Sorted array using Merge Sort: " + Arrays.toString(arr1));

        int[] arr2 = {38, 27, 43, 3, 9, 82, 10};
        System.out.println("Original array for Quick Sort: " + Arrays.toString(arr2));
        quickSort(arr2, 0, arr2.length - 1);
        System.out.println("Sorted array using Quick Sort: " + Arrays.toString(arr2));
    }
}
```

5. TEST RESULT / OUTPUT



6. SUMMARY:

This task demonstrates the implementation of Merge Sort and Quick Sort on an array in Java. Merge Sort is a reliable and efficient sorting algorithm, with a time complexity of O(n log n) in the worst case. Quick Sort, on the other hand, uses a pivot element and partitions the array, recursively sorting subarrays.